



DEFECT DETECTION SYSTEM

Code Analysis & Bug Report

Comparing `defect_analysis_v2_2.py` (Original) vs `defects_final_main.py` (Updated)

2 Python Files Analysed	3 New GUI Features	4 Bugs Found	4 Fixes Applied
---	--	--	---

Table of Contents

1.	Overview — What These Files Are	3
2.	File 1: Original Stable Version (defect_analysis_v2_2.py)	3
3.	File 2: Updated Version (defects_final_main.py)	4
4.	What Changed: Side-by-Side Comparison	5
5.	The Batch Processing Pipeline — How It Works	6
6.	Root Cause: Why The New Code Gives Wrong Results	7
7.	Bug 1 — Parameter Key Name Mismatch	8
8.	Bug 2 — Stale high_thresh Variable	9
9.	Bug 3 — Perspective Warp Applied By Default	10
10.	Bug 4 — Batch Starts Without Config	11
11.	All Fixes Explained	12
12.	Quick Reference: Old Key Names vs New Key Names	13

1. Overview — What These Files Are

This report compares two Python application files used for detecting defects in additive manufacturing (3D printing) images. Both files do the same core job — they analyse images using a technique called **Watershed Segmentation** to find defect regions — but they have different GUI designs. The problem is that the updated GUI version introduced bugs in the Batch Processing module that cause wrong defect/non-defect classification.

[ORIGINAL] defect_analysis_v2_2.py	67 KB
The stable, working version. Uses a traditional tabbed notebook GUI.	
[UPDATED] defects_final_main.py	135 KB
The new version with a modern dark-themed GUI. Contains batch processing bugs.	

2. File 1 — Original Stable Version

defect_analysis_v2_2.py

Think of this as a well-tested factory machine. It has three rooms (tabs) and does exactly what it's supposed to do. Every setting is tuned, every step is numbered correctly, and the batch processing output matches what you see in single-image mode.

TAB 1	ROI Selection You click on an image to draw a polygon (Region of Interest). The polygon marks the area where defects should be detected. Points are saved to a JSON file.
TAB 2	Watershed Analysis (Single Image) A 10-step pipeline: Mask → Grayscale → Sobel Edge → Markers → Watershed → Morph Close → Morph Open → Fill Holes → Label → Final Result. Each step can be tuned with sliders.
TAB 3	Batch Processing Reads a config JSON (containing the ROI polygon + all pipeline parameters), loops over every image in a folder, runs the same pipeline, and saves each image to either a "defects" or "non_defects" subfolder.

Pipeline Steps (Original — 10 Steps)

Step 1	ROI Mask	Black out everything outside the polygon
Step 2	Grayscale	Convert to single-channel for processing
Step 3	Sobel Gradient	Find edges — creates "elevation map"
Step 4	Histogram Markers	Label background (dark) and foreground (bright) pixels
Step 5	Watershed	Flood-fill from markers using elevation map

Step 6	Morph Close	Fill small gaps in detected regions
Step 7	Morph Open	Remove noise and small blobs
Step 8	Fill Holes	Fill enclosed holes inside regions
Step 9	Label Components	Number each separate region
Step 10	Final Result	Draw red overlay on valid defect regions

Config File Structure (original): The pipeline parameters are stored under keys named exactly after the steps: `step_4_markers`, `step_6_morph_close`, `step_10_final`, etc.

3. File 2 — Updated Version

defects_final_main.py

This file kept the same backend image processing logic but wrapped it in a completely new GUI. New navigation buttons replaced the old tabs, a dark steel colour theme was added, a login screen was introduced, and two brand-new pipeline steps were inserted. These changes are purely cosmetic and workflow improvements — but they accidentally broke the batch processing.

+ Dark Theme GUI	Professional dark colour scheme with a header bar, styled buttons, and scrollable parameter panels.
+ Login Screen	A splash screen + login window protects the Single Image tab (username: admin / password: admin). Batch Processing is always visible.
+ Navigation Pills	Step number buttons [0][1]...[12] let you jump to any pipeline step directly without clicking Next many times.
+ Inline ROI Drawing	Instead of Tab 1 being a separate ROI window, Step 1 of the pipeline IS the ROI drawing step on the main canvas.
+ Step 2 — Rotate/Flip (NEW)	A brand new step was inserted at position 2. You can rotate the masked image by any angle and flip it horizontally or vertically before analysis.
+ Step 12 — Perspective Warp (NEW)	Another new step at the end. It warps the ROI quad into a flat 272×272 px top-down view, useful for physical panel inspection.
+ Auto-save Config	When you close an ROI polygon, the config is auto-saved to the image folder AND to configurations/conf.json (which the Batch tab auto-loads on startup).
+ Output Folder Choice	Batch Processing now lets you pick a separate output folder instead of always writing inside the input folder.

Pipeline Steps (Updated — 13 Steps)

Step 0	Original Image	Display original loaded image — no processing
Step 1	ROI Drawing	Click on canvas to draw the polygon interactively
Step 2	Rotate / Flip NEW	NEW — rotate or mirror the masked image
Step 3	Grayscale	Convert to single-channel
Step 4	Sobel Gradient	Edge map
Step 5	Hist. Markers	Background/foreground markers
Step 6	Watershed	Flood-fill segmentation
Step 7	Morph Close	Fill gaps

Step 8	Morph Open	Remove noise
Step 9	Fill Holes	Enclosed-hole fill
Step 10	Label Components	Number regions
Step 11	Final Result	Red overlay on defects
Step 12	Perspective Warp NEW	NEW — top-down rectification

5. The Batch Processing Pipeline — How It Works

When you click **Start Process** in the Batch tab, the app loops over every image file in the input folder and processes each one through a condensed version of the same Watershed pipeline used in Single Image mode. Here is the flow, step by step:

1	Load Image	cv2.imread() reads each image as a BGR numpy array.
2	Get ROI Points	Reads polygon_points from the config JSON. If no config, falls back to full image corners.
3	Apply ROI Mask	Everything outside the polygon is blacked out (set to 0,0,0).
4	Rotate / Flip	Optional: rotate by angle, flip horizontally or vertically.
5	Grayscale	Convert BGR to single-channel grayscale for edge detection.
6	Sobel Gradient	Compute X and Y derivatives, combine as $\sqrt{Gx^2+Gy^2}$. This is the "elevation map".
7	Histogram Markers	Dark pixels ($< \text{low_threshold}$) $\rightarrow$ background markers. Bright pixels ($> \text{high_threshold}$) $\rightarrow$ foreground markers.
8	Watershed	cv2.watershed() floods from the markers using the elevation map. Boundaries get label -1.
9	Binary Mask	Pixels with watershed label $> \text{high_threshold}$ are treated as defect foreground.
10	Morph Close	Dilate then erode to close small holes and connect nearby blobs.
11	Morph Open	Erode then dilate to remove speckle noise.
12	Fill Holes	scipy.ndimage.binary_fill_holes() fills enclosed cavities.
13	Label Components	scipy.ndimage.label() gives each connected region a unique number. Small regions are removed.
14	Count Defects	For each labelled region, find its contour area. Count regions where $\text{min_area} \leq \text{area} \leq \text{max_area}$.
15	Save & Classify	If $\text{valid_segments} \geq 1 \rightarrow$ save to "defects" folder. Else $\rightarrow$ "non_defects" folder.

6. Root Cause — Why The New Code Gives Wrong Results

Simple Analogy to Understand the Root Cause

Imagine a factory machine (the pipeline) controlled by a settings sheet (the config JSON). The original machine had dials labelled "Dial 4", "Dial 6", "Dial 10". The new machine added two extra dials, so every label shifted up by one: "Dial 4" is now "Dial 5", "Dial 6" is now "Dial 7", "Dial 10" is now "Dial 11".

When the new machine reads the old settings sheet, it looks for "Dial 5" but the sheet only has "Dial 4". It finds nothing, and silently uses a factory default instead. All your carefully tuned settings are ignored. The machine still runs — but with completely wrong settings — which is why defect/non-defect results are inconsistent.

The core issue is that when two new steps (Rotate/Flip at position 2 and Perspective Warp at position 12) were added to the updated app, all existing step keys in the config file format were shifted up by one. But config files saved by the original app still use the old numbering. When the batch module loads an old config, it reads the wrong (or missing) keys and silently falls back to hardcoded defaults — giving results that differ from single-image mode.

Key Renaming at a Glance

Old Config Key	New Pipeline Reads	Result If Old Config Used
step_3_sobel	step_4_sobel	Sobel step uses default (enabled=True) – usually OK
step_4_markers	step_5_markers	CRITICAL – high_threshold reverts to 150 default
step_5_watershed	step_6_watershed	Watershed uses default (enabled=True) – usually OK
step_6_morph_close	step_7_morph_close	Iterations reset to 2 (your tuned value lost)
step_7_morph_open	step_8_morph_open	Iterations reset to 2 (your tuned value lost)
step_8_fillholes	step_9_fillholes	Fill holes uses default (enabled=True) – usually OK
step_9_labeling	step_10_labeling	min_size reset to 0 (your tuned value lost)
step_10_final	step_11_final	CRITICAL – min_area and max_area reset to 0/10000

7–10. Individual Bugs — Detailed Breakdown

BUG 1 — Parameter Key Name Mismatch

WHERE: `tab3_process_single_image()` — the line: `params = self.tab3_config_data['pipeline_params']`

PROBLEM: The updated pipeline uses renamed keys (`step_5_markers`, `step_11_final`, etc.) but config files saved by the original app still use old keys (`step_4_markers`, `step_10_final`). When batch loads an old config, it cannot find the new key names and silently falls back to hardcoded defaults for every parameter.

IMPACT: ALL tuned parameters (thresholds, iterations, area limits) are ignored. Every image is processed with default settings instead of your calibrated values. Results vary wildly from single-image mode.

Example — what the old config looks like vs what the new code expects:

Old Config JSON	New Code Reads
<pre>"pipeline_params": { "step_4_markers": { "low_threshold": 30, "high_threshold": 120 }, "step_10_final": { "min_area": 50, "max_area": 5000 } }</pre>	<pre>params.get("step_5_markers") → Returns {} because key "step_5_markers" not found! Falls back to default 150. params.get("step_11_final") → Returns {} because key "step_11_final" not found! Falls back to 0 / 10000.</pre>

BUG 2 — Stale high_thresh Variable

WHERE: In `tab3_process_single_image()` just before the Watershed markers block

PROBLEM: The variable `high_thresh = 150` is set as a "fallback" BEFORE the if-block that should update it. If `step_5_markers` key is missing from the config (because it's an old config), the if-block body runs but uses `params['step_5_markers']` which raises a `KeyError` or returns wrong values. Later, `binary_seg = (segmentation > high_thresh)` uses the stale 150.

IMPACT: The binary mask that separates defect pixels from background pixels is computed with the wrong threshold. If your real `high_threshold` was 120, the system uses 150 instead — potentially classifying defect pixels as background, causing images to be put in the wrong folder.

The exact buggy code pattern:

```
# BUGGY CODE: high_thresh = 150 # ← set here, before the if-block if  
params.get('step_5_markers', {}).get('enabled', True): low_thresh =  
params['step_5_markers'].get('low_threshold', 30) high_thresh =  
params['step_5_markers'].get('high_threshold', 150) # ^ if 'step_5_markers' key is missing from
```

```
old config, # this entire block is SKIPPED because .get() returns {} # which has
.get('enabled') = None = falsy... OR # the key IS there but wrong values come through. # Later:
binary_seg = (segmentation > high_thresh) # ← uses stale 150!
```

BUG 3 — Perspective Warp Applied By Default

WHERE: In `tab3_process_single_image()` near the end, the line: `warp_enabled = params.get(..., True)`

PROBLEM: The Perspective Warp (Step 12) is a new feature that does not exist in old configs. When a key is missing from a dictionary, Python's `.get()` returns the default value. The code uses `.get("step_12_perspective", {}).get("enabled", True)` — note the default is `True`. So every image processed with an old config gets perspective-warped, even though the user never asked for it.

IMPACT: Every output image is cropped and distorted to a 272×272 pixel square. The red defect contours drawn before the warp are also warped. An image that had 2 defect regions might show 0 after warping if they get cropped out — causing it to be misclassified as non-defect.

The one-character-fix that causes the problem:

```
# BUGGY: warp_enabled = params.get('step_12_perspective', {}).get('enabled', True) # ^^^^ #
Default is True → warp fires for ALL old configs! # FIXED: warp_enabled =
params.get('step_12_perspective', {}).get('enabled', False) # ^^^^^ # Default is False → warp
only fires if explicitly enabled
```

BUG 4 — Batch Starts Without a Config

WHERE: `tab3_check_ready()` method — the readiness check logic

PROBLEM: The original app required both an input folder AND a valid config (with `polygon_points` and `pipeline_params`) before enabling the Start button. The updated version simplified this check and allows the Start button to be enabled with only an input folder selected — no config needed.

IMPACT: If the user accidentally clicks Start without loading a config, the pipeline runs with a full-image ROI (all four corners) and completely default parameters. This silently produces wrong results with no warning. The original gave a clear error message instead.

Original	Updated (New)
Original <code>check_ready</code> logic Requires: <code>input_folder</code> AND config file with <code>polygon_points</code> AND <code>pipeline_params</code>. If either is missing → button stays disabled.	Updated <code>check_ready</code> logic (buggy) Requires: only <code>input_folder</code>. If only folder is selected → button becomes ENABLED. No config = uses default ROI + default params silently.

11. All Fixes — What Was Done and Why

Four fixes were applied to `defects_final_main.py`. The fixes are surgical — they touch only the batch processing module and do not change the GUI, the Single Image pipeline, or any other feature. The goal is to make batch results identical to Single Image results.

FIX 1 — `_normalize_params()` — New Helper Method

WHAT: A new method was added to the class that translates old-style config key names into the new key names before the batch pipeline uses them. It also injects safe defaults for `step_2_rotate_flip` (disabled) and `step_12_perspective` (disabled) if they are missing.

WHY: This is the single most important fix. It makes the batch pipeline backward-compatible with any config file ever saved, whether by the original app or the new one. It is called once per image, at the top of the params-loading block.

What `_normalize_params()` does internally:

```
KEY_MAP = { 'step_3_sobel': 'step_4_sobel', # renamed 'step_4_markers': 'step_5_markers', #
renamed (CRITICAL) 'step_5_watershed': 'step_6_watershed', # renamed 'step_6_morph_close':
'step_7_morph_close', # renamed 'step_7_morph_open': 'step_8_morph_open', # renamed
'step_8_fillholes': 'step_9_fillholes', # renamed 'step_9_labeling': 'step_10_labeling', #
renamed 'step_10_final': 'step_11_final', # renamed (CRITICAL) } # Renames old keys → new keys
# Adds missing step_2_rotate_flip: {enabled: False} # Adds missing step_12_perspective:
{enabled: False}
```

FIX 2 — Apply `_normalize_params()` When Loading Config

WHAT: In `tab3_process_single_image()`, changed: `params = self.tab3_config_data['pipeline_params']` to: `params = self._normalize_params(self.tab3_config_data['pipeline_params'])`

WHY: Without this call, the translation never happens. Fix 1 (the helper method) alone does nothing — Fix 2 is the wiring that connects the normalizer to the actual pipeline run. A debug print is also added so you can verify the correct keys are being used.

FIX 3 — Change Perspective Warp Default from True to False

WHAT: Changed the default value in: `params.get('step_12_perspective', {}).get('enabled', True)` to: `params.get('step_12_perspective', {}).get('enabled', False)`

WHY: This is a double-protection alongside Fix 1. Even if somehow `_normalize_params()` is not called or is bypassed, the warp will not fire unless the config explicitly enables it. This is the correct design: new features should be opt-in, not opt-out.

FIX 4 — Fix Stale `high_thresh` + Restore Strict `tab3_check_ready()`

WHAT: Two sub-fixes: (a) Replaced the stale `high_thresh = 150` fallback pattern with a safe lookup: `_marker_cfg = params.get("step_5_markers", params.get("step_4_markers", {}))` that reads the value before the if-block. (b) Restored `tab3_check_ready()` to require `input_folder` AND a config with both `polygon_points` and `pipeline_params` before enabling the Start button.

WHY: (a) Ensures the threshold used for `binary_seg` is always the correct value from config, never a stale default. (b) Prevents silent wrong-results caused by accidentally running batch without a config.

Fix Summary

#	Fix	Bugs It Solves	Risk
Fix 1	<code>_normalize_params()</code> method	Bug 1, Bug 2 (partially)	None — new method, not called elsewhere
Fix 2	Wire normalizer to batch pipeline	Bug 1 (complete), Bug 3 (via injected disabled key)	None — 1 line change
Fix 3	Warp default False	Bug 3 (complete)	None — only changes default for missing keys
Fix 4a	Fix stale <code>high_thresh</code>	Bug 2 (complete)	None — logic-equivalent, more robust
Fix 4b	Strict <code>tab3_check_ready</code>	Bug 4	None — restores original behaviour

12. Quick Reference

Complete Key Name Translation Table

Use this table to manually convert any old config file to the new format if needed.

Old Key (Original App)	New Key (Updated App)	What It Controls
step_2_grayscale	step_3_grayscale	Enable/disable grayscale conversion
step_3_sobel	step_4_sobel	Enable/disable Sobel gradient edge detection
step_4_markers (low_threshold, high_threshold)	step_5_markers	Background/foreground threshold values – CRITICAL
step_5_watershed	step_6_watershed	Enable/disable watershed algorithm
step_6_morph_close (iterations)	step_7_morph_close	Number of morphological closing iterations
step_7_morph_open (iterations)	step_8_morph_open	Number of morphological opening iterations
step_8_fillholes	step_9_fillholes	Enable/disable binary hole filling
step_9_labeling (min_size)	step_10_labeling	Minimum region size in pixels to keep
step_10_final (min_area, max_area)	step_11_final	Valid contour area range – CRITICAL
(did not exist)	step_2_rotate_flip	NEW: rotate angle, flip H/V – disabled by default
(did not exist)	step_12_perspective	NEW: perspective warp – disabled by default

Debugging Checklist — If Results Are Still Inconsistent

- Check the terminal for the debug print: `[DEBUG] Normalized params keys: [...]` — verify it shows new-style key names (`step_5_markers`, not `step_4_markers`).
- Confirm your config JSON was saved by the UPDATED app (will have `step_5_markers`, `step_11_final`) or the ORIGINAL app (`step_4_markers`, `step_10_final`). Both are now handled.
- Verify `step_12_perspective` is either absent (safe — defaults to disabled) or explicitly set to `{"enabled": false}` in your config.
- Make sure `tab3_check_ready()` prevents the Start button from activating without a valid config loaded.
- Run a single known-defect image and known-non-defect image through both Single Image mode and Batch mode. The classification should match.
- If you add more pipeline steps in future, update the `KEY_MAP` inside `_normalize_params()` and ensure any new step defaults to disabled.

This report was generated automatically by the code analysis tool. The fixed file is: `defects_final_main_fixed.py`